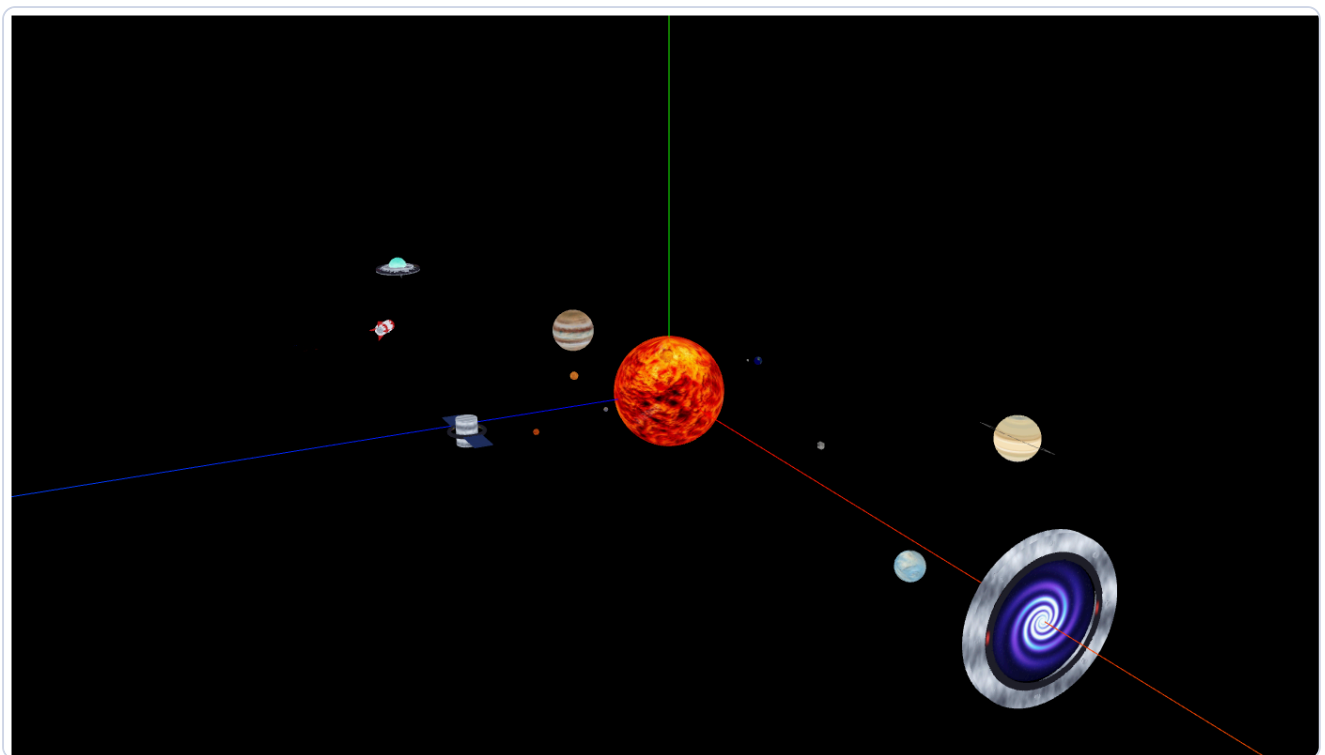


Simulador 3D del Sistema Solar

Guía de funcionamiento y explicación del código

Proyecto con three.js



Vista general del simulador funcionando

1. ¿Qué hace este proyecto?

Es un simulador en 3D que se ve en el navegador. Muestra el **Sol** quieto en el centro y los planetas **girando a su alrededor**. Además hay objetos contruidos por nosotros con las figuras básicas de three.js: un **cohete** (Cono), un **OVNI** (Cilindro), una **puerta estelar** (Ring) y un **satélite** (Torus).

Todo está hecho con **three.js** (<https://threejs.org>), una librería de JavaScript que dibuja gráficos 3D usando la tarjeta gráfica (WebGL). No hay que instalar nada: solo abrir `index.html`.

Estructura de carpetas

Carpeta / archivo	Para qué sirve
<code>nbproject/</code>	Archivos que NetBeans necesita para reconocer el proyecto.
<code>test/</code>	Carpeta para pruebas (vacía por ahora).
<code>public_html/index.html</code>	Página web. Carga las librerías y nuestro programa.
<code>public_html/Universo.js</code>	Nuestro código. Aquí se crea todo el sistema solar.
<code>public_html/three.js</code>	La librería three.js (no se modifica).
<code>public_html/OrbitControls.js</code>	Permite mover la cámara con el mouse.
<code>public_html/GLTFLoader.js</code>	Permite cargar modelos 3D externos (<code>nave.glb</code>).
<code>public_html/tex_planetas/</code>	Todas las imágenes (texturas) que se pegan sobre los objetos.

Cómo ejecutarlo en NetBeans

1. **File → Open Project** y elegir la carpeta interna `SistemaSolar/SistemaSolar` (la que tiene `nbproject`).
2. Clic derecho sobre `public_html/index.html` → **Run File**.

Controles: arrastrar con el mouse = girar la cámara; rueda = acercar/alejar; clic derecho y arrastrar = desplazar la vista.

2. Conceptos básicos de three.js

Para entender el código hay que conocer estas piezas. Piensa en una película:

Concepto	Comparación	En el código
Scene (escena)	El set de filmación donde se pone todo.	<code>escena</code>

Camera (cámara)	Desde dónde se mira.	<code>camara</code>
Renderer	El que “toma la foto” de la escena y la dibuja en pantalla.	<code>renderer</code>
Geometry (geometría)	La forma: esfera, cono, cilindro...	<code>SphereGeometry</code> , <code>ConeGeometry</code> ...
Material	Cómo se ve la superficie (color, imagen).	<code>MeshBasicMaterial</code>
Texture (textura)	La imagen que se pega sobre la forma.	<code>textura('texturaSol1.jpg')</code>
Mesh	Un objeto visible = geometría + material.	<code>new THREE.Mesh(geometria, material)</code>
Group (grupo)	Una caja donde se juntan varios objetos para moverlos como uno solo.	<code>grupoSaturno</code> , <code>grupoCono</code> ...

Posición, rotación y escala

Todo objeto tiene tres propiedades: `position` (dónde está: `x` derecha/izquierda, `y` arriba/abajo, `z` adelante/atrás), `rotation` (cuánto está girado, en *radianes*; `Math.PI` = media vuelta) y `scale` (qué tan grande es; 2 = el doble).

En pantalla, los ejes se dibujan con colores: **X rojo**, **Y verde**, **Z azul** (`AxesHelper`).

3. El código explicado por partes

3.1 Preparar el escenario

```
var escena, renderer, controls;

renderer = new THREE.WebGLRenderer({antialias: true});
renderer.setSize(window.innerWidth, window.innerHeight);
document.body.appendChild(renderer.domElement);

//crear escena
escena = new THREE.Scene();
escena.add(new THREE.AxesHelper(5000));
```

- `WebGLRenderer` crea el “lienzo” donde se dibuja. `setSize` lo hace del tamaño de la ventana.
- `appendChild` pone ese lienzo dentro de la página web.
- `Scene` es la escena vacía; `AxesHelper` le agrega las líneas de los ejes.

```
//Camara -->          angulo de vision | relacion de aspecto | alcance min | alcance
max
var camara = new THREE.PerspectiveCamera(30, window.innerWidth / window.innerHeight, 0.1, 100000);
camara.position.set(4300, 1600, 2200);

window.addEventListener('resize', function(){
    camara.aspect = window.innerWidth / window.innerHeight;
    camara.updateProjectionMatrix();
    renderer.setSize(window.innerWidth, window.innerHeight);
});
```

- `PerspectiveCamera(30, aspecto, 0.1, 100000)`: 30° de ángulo de visión, la proporción de la ventana, y lo más cerca y lo más lejos que puede ver.
- `camara.position.set(0, 1200, 2800)`: la cámara está arriba (1200) y atrás (2800) para ver todo el sistema.
- El bloque `resize` ajusta la cámara si cambias el tamaño de la ventana.

3.2 Función para cargar texturas

```
var cargador = new THREE.TextureLoader();
function textura(ruta){
    return cargador.load('tex_planetas/' + ruta);
}
```

Es un atajo: `textura('texturaSol1.jpg')` carga la imagen desde la carpeta `tex_planetas`. Así no se repite la ruta en cada objeto.

3.3 El Sol y los planetas

```
//sol (quieto en el centro, no rota ni se mueve)
var sol = new THREE.Mesh(new THREE.SphereGeometry(200, 64, 48), new THREE.MeshBasicMaterial({map:
textura('texturaSol1.jpg')}));
sol.position.set(0, 0, 0);
```

Cada planeta es una **esfera** (`SphereGeometry(radio, segmentosX, segmentosY)`) con una textura. Los segmentos son cuántas “tiras” forman la esfera: más segmentos = más redonda.

Cuerpo	Radio	Distancia al Sol (órbita)
Sol	200	0 (en el centro, quieto)
Mercurio	8	300
Venus	15.4	400
Tierra	16	510
Marte	10.3	620
Júpiter	86	900
Saturno	77	1250

Urano	42	1600
-------	----	------

Las medidas **no son reales**: el orden de tamaños sí se respeta (Júpiter el mayor, Mercurio el menor), pero se suavizaron para que los planetas pequeños se vean en pantalla. A escala real serían puntos invisibles.

3.4 Tierra y Luna (un grupo)

```
//tierra + luna (la luna orbita a la Tierra, por eso van en un grupo)
var Tierra = new THREE.Mesh(new THREE.SphereGeometry(16, 32, 24), new
THREE.MeshBasicMaterial({map: textura('texturaTierra1.jpg')}));
var luna = new THREE.Mesh(new THREE.SphereGeometry(4.5, 32, 24), new THREE.MeshBasicMaterial({map:
textura('texturaLuna1.jpg')}));
var grupoTierra = new THREE.Group();
grupoTierra.add(Tierra);
grupoTierra.add(luna);
```

La Luna y la Tierra se meten en un `Group`. Así, cuando movemos el grupo alrededor del Sol, la Luna viaja con la Tierra, y por separado la Luna da vueltas alrededor de la Tierra.

3.5 Saturno y su anillo (un grupo)

```
//saturno + anillo
var Saturno = new THREE.Mesh(new THREE.SphereGeometry(77, 48, 32), new
THREE.MeshBasicMaterial({map: textura('texturasaturno1.jpg')}));
var anilloSaturno = new THREE.Mesh(
    new THREE.TorusGeometry(115, 17, 2, 96),
    new THREE.MeshBasicMaterial({map: textura('texturaAnilloSaturno.jpg'), side:
THREE.DoubleSide}));
anilloSaturno.rotation.x = Math.PI / 2 - 0.45;
anilloSaturno.scale.z = 0.15;
var grupoSaturno = new THREE.Group();
grupoSaturno.add(Saturno);
grupoSaturno.add(anilloSaturno);
```

El anillo es un `TorusGeometry` (una dona) aplastado: `scale.z = 0.15` lo vuelve casi plano, y `rotation.x` lo inclina. Esfera + anillo se agrupan en `grupoSaturno`.

3.6 Asteroide y nave

```
//asteroide (entre Marte y Jupiter)
var Asteroide = new THREE.Mesh(new THREE.DodecahedronGeometry(14), new
THREE.MeshBasicMaterial({map: textura('texturaAst.png')}));
Asteroide.position.set(760, 15, -160);

//NAVE
var loader1 = new THREE.GLTFLoader();
var navecruzero;
loader1.load(
    'nave.glb',
    function(objeto){
        navecruzero = objeto.scene;
        navecruzero.position.set(-900, 100, 1100);
        navecruzero.scale.set(5, 5, 5);
        escena.add(navecruzero);
    },
    undefined,
    function(error){
        console.log(error);
    }
);
```

El asteroide es un `DodecahedronGeometry` (figura de 12 caras) quieto entre Marte y Júpiter. La nave es un modelo ya hecho (`nave.glb`) que se carga con `GLTFLoader` ; la función de adentro se ejecuta cuando termina de cargar y la agrega a la escena.

4. Las figuras que construimos

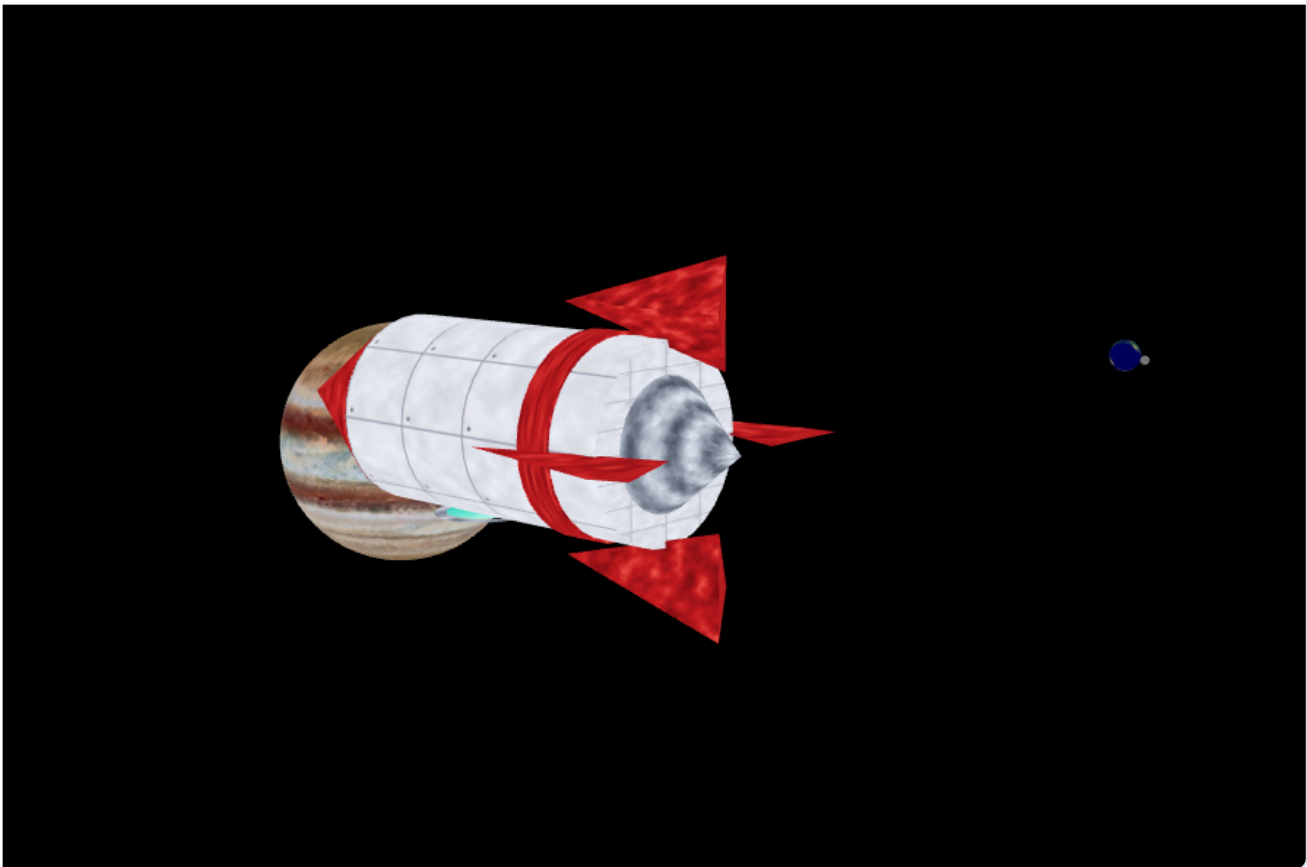
Cada una usa una figura básica de three.js y se arma como un **grupo** de piezas, igual que Saturno. Las piezas se colocan con `position` relativa al centro del grupo.

4.1 Materiales con imagen

```
function material(ruta){
    return new THREE.MeshBasicMaterial({map: textura(ruta), side: THREE.DoubleSide});
}
var texCohete = material('texturaCohete.jpg');
var texCoheteRojo = material('texturaCoheteRojo.jpg');
var texCristal = material('texturaCristal.jpg');
var texOvni = material('texturaOvni.jpg');
var texLuces = material('texturaLuces.jpg');
var texCilindro = material('texturaCilindro.jpg');
var texPanel = material('texturaPanel.jpg');
```

`material('archivo.jpg')` crea un material con la imagen indicada. `DoubleSide` hace que se vea por las dos caras (necesario en piezas planas como `RingGeometry`).

4.2 Cohete — ConeGeometry (Cono)



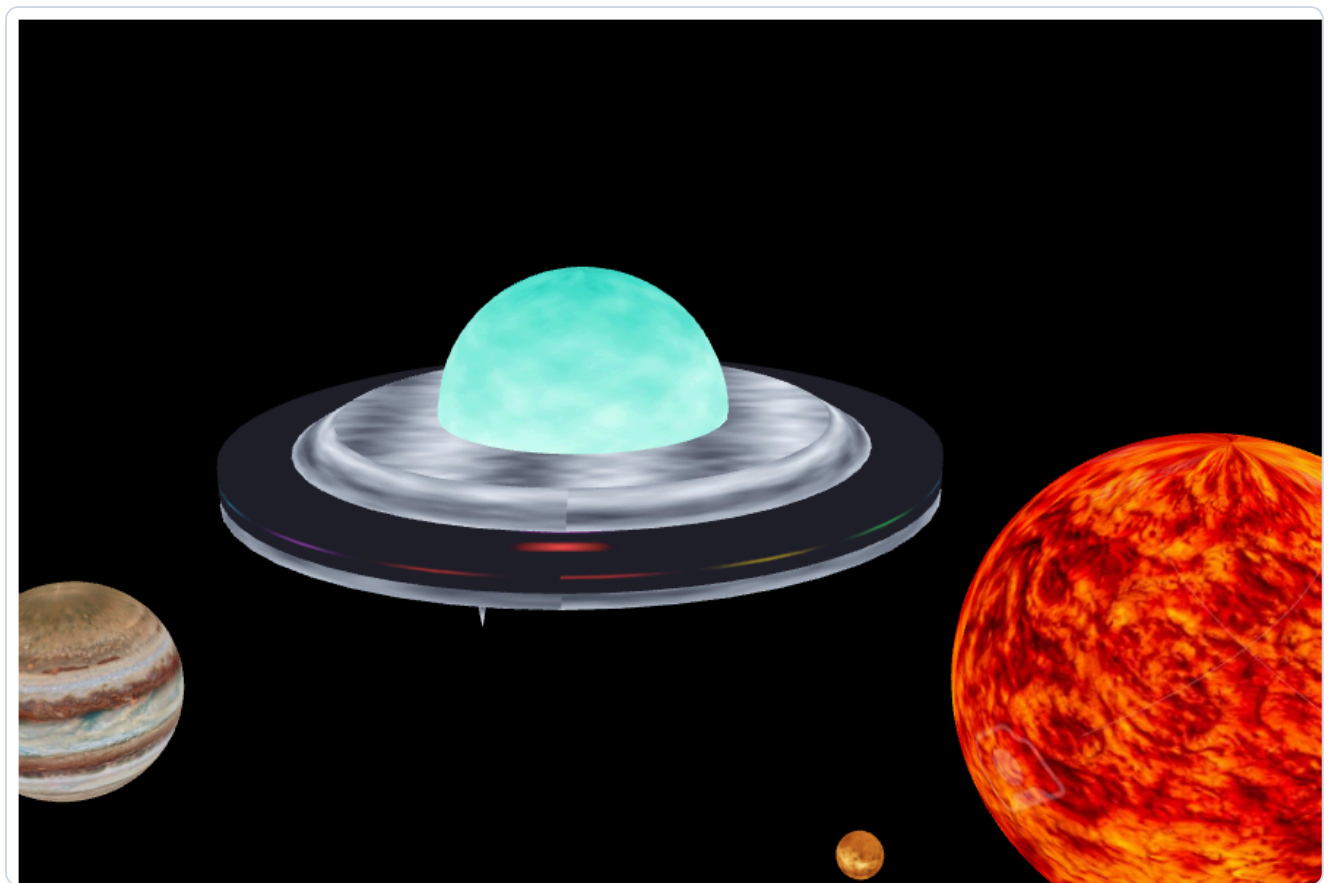
El cohete (visto de lado porque vuela)

```
//CONO -> COHETE: cuerpo, punta, aletas, tobera, ventana
var grupoCono = new THREE.Group();
var cuerpoCohete = new THREE.Mesh(new THREE.CylinderGeometry(0.9, 0.9, 3.2, 32), texCohete);
var franjaCohete = new THREE.Mesh(new THREE.CylinderGeometry(0.93, 0.93, 0.35, 32),
texCoheteRojo);
franjaCohete.position.y = -0.9;
var puntaCohete = new THREE.Mesh(new THREE.ConeGeometry(0.9, 1.6, 32), texCoheteRojo);
puntaCohete.position.y = 2.4;
var toberaCohete = new THREE.Mesh(new THREE.ConeGeometry(0.6, 0.8, 24), tex0vni);
toberaCohete.rotation.x = Math.PI;
toberaCohete.position.y = -2;
var ventanaCohete = new THREE.Mesh(new THREE.RingGeometry(0.35, 0.47, 32), tex0vni);
ventanaCohete.position.set(0, 0.7, 0.92);
var vidrioCohete = new THREE.Mesh(new THREE.CircleGeometry(0.35, 32), texCristal);
vidrioCohete.position.set(0, 0.7, 0.91);
grupoCono.add(cuerpoCohete, franjaCohete, puntaCohete, toberaCohete, ventanaCohete, vidrioCohete);
for (var k = 0; k < 4; k++){
    var ang = k * Math.PI / 2;
    var aleta = new THREE.Mesh(new THREE.ConeGeometry(0.55, 1.8, 3), texCoheteRojo);
    aleta.scale.z = 0.12;
    aleta.position.set(Math.cos(ang) * 1.15, -1.3, Math.sin(ang) * 1.15);
    aleta.rotation.y = -ang;
    grupoCono.add(aleta);
}
```

Pieza	Geometría	Función
Cuerpo y franja	Cilindro	Tubo blanco con una banda roja.

Punta	Cono	Se coloca encima del cuerpo ($y = 2.4$).
Tobera	Cono	Cono girado Math.PI (volteado) abajo.
Ventana	Ring + Círculo	Marco y vidrio.
4 aletas	Cono de 3 lados	Un <code>for</code> las reparte cada 90° ($\text{Math.PI} / 2$) con <code>cos</code> y <code>sin</code> , y se aplastan con <code>scale.z</code> .

4.3 OVNI — CylinderGeometry (Cilindro)

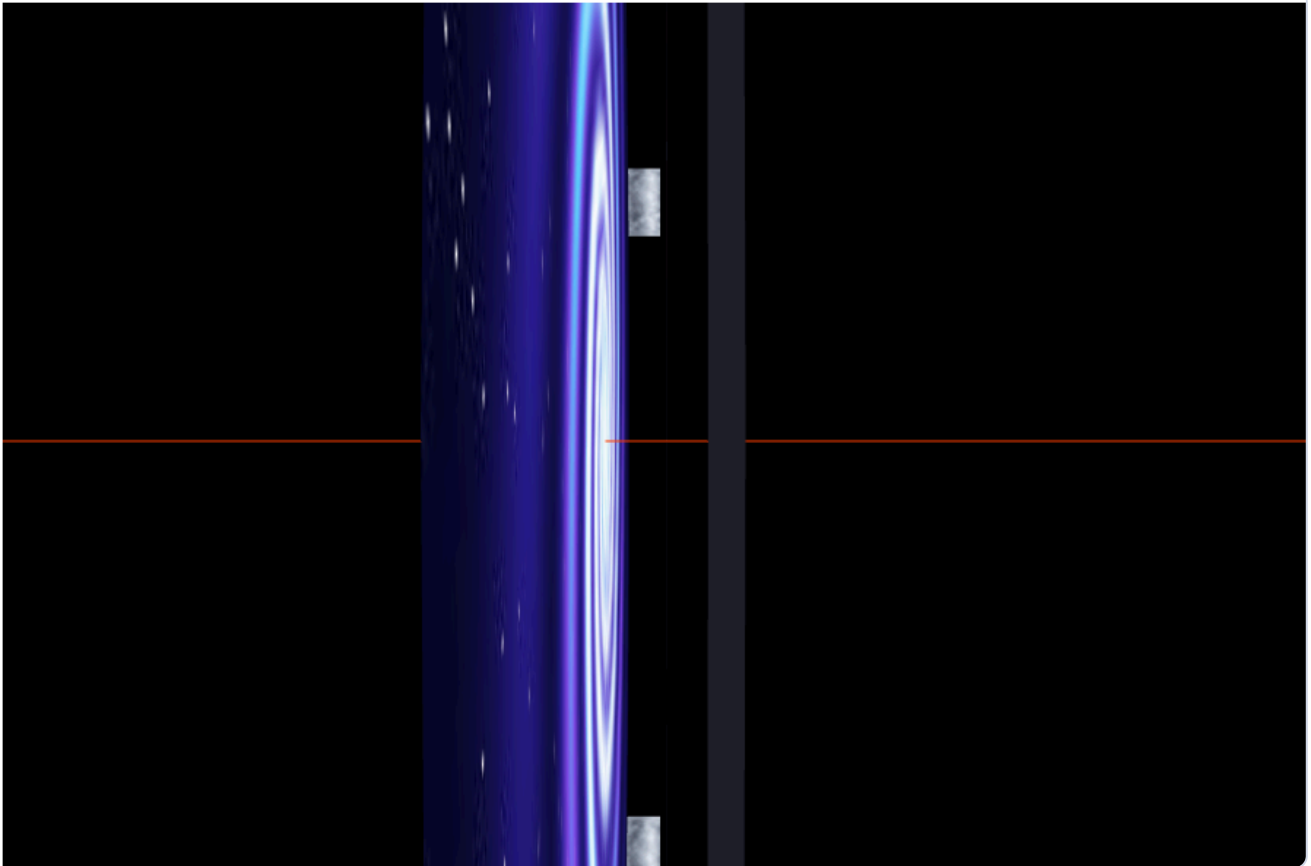


EI OVNI

```
//CILINDRO -> OVNI: platillo, cupula de cristal, aro de luces y patas
var grupoCilindro = new THREE.Group();
var platillo = new THREE.Mesh(new THREE.CylinderGeometry(2.2, 3.2, 0.6, 48), texOvni);
var cupula = new THREE.Mesh(new THREE.SphereGeometry(1.3, 32, 16, 0, Math.PI * 2, 0, Math.PI / 2),
texCristal);
cupula.position.y = 0.3;
var aroLuces = new THREE.Mesh(new THREE.CylinderGeometry(3.2, 3.2, 0.25, 64), texLuces);
aroLuces.position.y = -0.05;
grupoCilindro.add(platillo, cupula, aroLuces);
for (var p = 0; p < 3; p++){
    var angP = p * Math.PI * 2 / 3;
    var pata = new THREE.Mesh(new THREE.ConeGeometry(0.15, 0.9, 12), texOvni);
    pata.rotation.x = Math.PI;
    pata.position.set(Math.cos(angP) * 1.6, -0.75, Math.sin(angP) * 1.6);
    grupoCilindro.add(pata);
}
```

`CylinderGeometry(radioArriba, radioAbajo, alto, segmentos)` : al poner radios distintos el cilindro se vuelve un platillo. La cúpula es media esfera (los últimos parámetros de `SphereGeometry` cortan la mitad). El aro de luces es otro cilindro bajito con una textura de lucecitas, y las 3 patas son conos pequeños volteados.

4.4 Puerta estelar — `RingGeometry (Ring)`

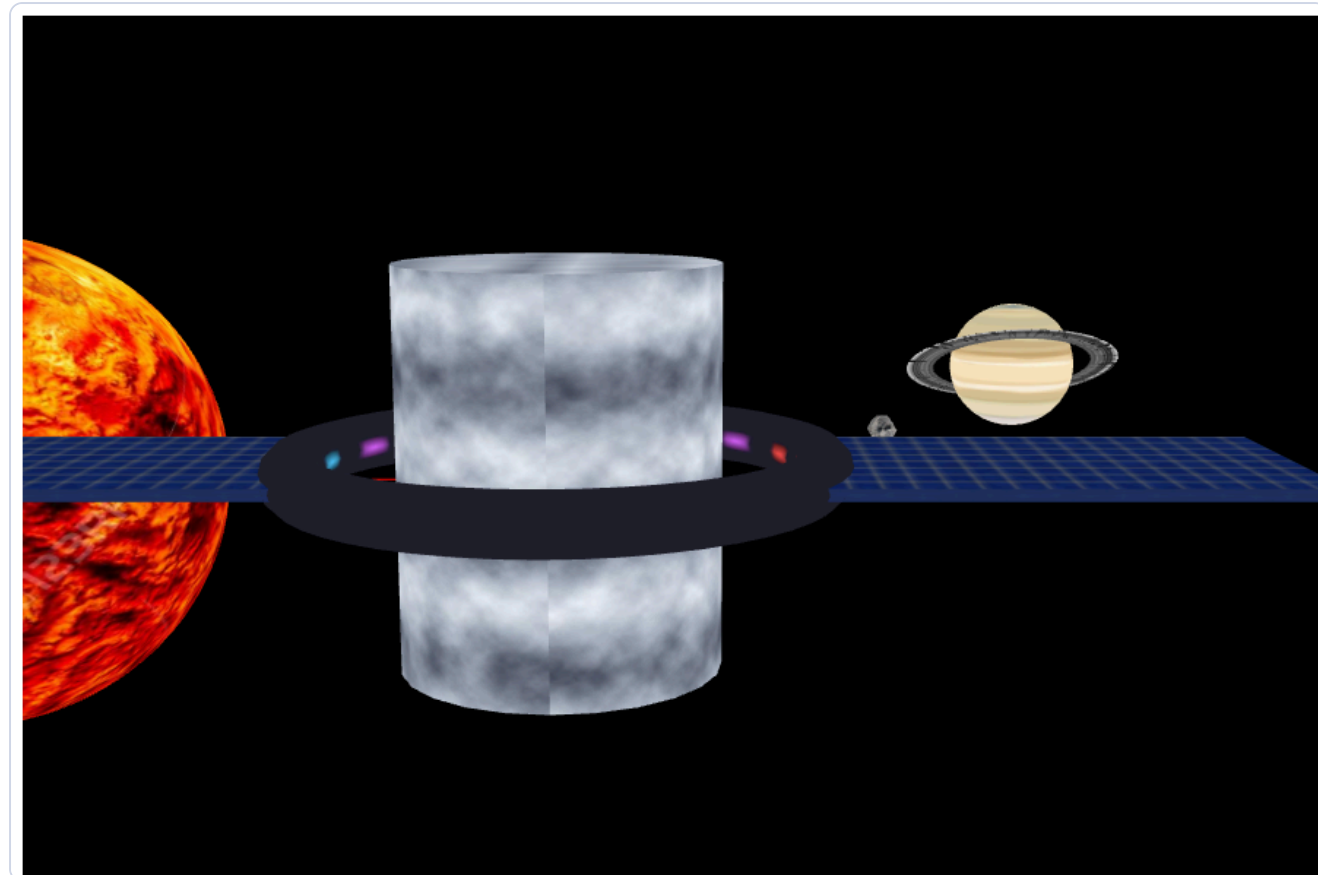


La puerta estelar

```
//RING -> PUERTA ESTELAR: marco metalico, anillo de luces y remolino de galaxia en el centro
var grupoRing = new THREE.Group();
var marcoPuerta = new THREE.Mesh(new THREE.RingGeometry(2.6, 3.3, 64), tex0vni);
var lucesPuerta = new THREE.Mesh(new THREE.RingGeometry(2.3, 2.6, 64), texLuces);
var remolino = new THREE.Mesh(new THREE.CircleGeometry(2.3, 64), material('texturaPortal.jpg'));
var marcoTrasero = marcoPuerta.clone();
marcoTrasero.position.z = -0.35;
lucesPuerta.position.z = 0.02;
remolino.position.z = -0.1;
grupoRing.add(marcoPuerta, marcoTrasero, lucesPuerta, remolino);
for (var q = 0; q < 8; q++){
  var angQ = q * Math.PI / 4;
  var puntal = new THREE.Mesh(new THREE.CylinderGeometry(0.12, 0.12, 0.35, 8), tex0vni);
  puntal.rotation.x = Math.PI / 2;
  puntal.position.set(Math.cos(angQ) * 2.95, Math.sin(angQ) * 2.95, -0.175);
  grupoRing.add(puntal);
}
grupoRing.rotation.y = Math.PI / 2; //de frente en X: la puerta mira hacia el Sol a lo largo del eje X
```

`RingGeometry(radioInterno, radioExterno, segmentos)` es un aro plano. Se usan dos como marco (uno atrás para dar grosor), uno más para las luces, y un `CircleGeometry` con una textura de galaxia en espiral al centro. Los 8 puntales (cilindros chiquitos) unen los dos marcos. `rotation.y = Math.PI / 2` la gira 90° para que su cara mire a lo largo del eje X.

4.5 Satélite — `TorusGeometry` (Torus)



El satélite

```
//TORUS GEOMETRY -> SATELITE: cuerpo, anillo (el unico Torus) y dos paneles solares
var grupoTorus = new THREE.Group();
var cuerpoSat = new THREE.Mesh(new THREE.CylinderGeometry(0.8, 0.8, 2, 24), texOvni);
var anilloSat = new THREE.Mesh(new THREE.TorusGeometry(1.3, 0.15, 12, 48), texLuces);
anilloSat.rotation.x = Math.PI / 2;
var panelSatA = new THREE.Mesh(new THREE.BoxGeometry(2.4, 0.06, 1.4), texPanel);
panelSatA.position.x = 2.4;
var panelSatB = panelSatA.clone();
panelSatB.position.x = -2.4;
grupoTorus.add(cuerpoSat, anilloSat, panelSatA, panelSatB);
```

`TorusGeometry(radio, grosorDelTubo, segmentosTubo, segmentosAnillo)` es una dona. Aquí es el anillo que rodea el cuerpo; se gira `Math.PI / 2` para que quede acostado. Los paneles solares son cajas (`BoxGeometry`) planas; el segundo se crea con `clone()` (copia del primero) y se pone al lado contrario con `x` negativo.

Resumen de lo pedido: Cono → cohete (vuela) · Cilindro → OVNI (vuela) · Ring → puerta estelar (quieta, al final del eje X) · TorusGeometry → satélite (vuela). El **TorusKnot** (cometa) se quitó por ahora.

4.6 Colocar las figuras en el espacio

```
//posicionar las formas alineadas sobre el eje X (misma altura Y y misma Z)
grupoCono.position.set(-680, 150, 900);      //cohete: vuela
grupoCilindro.position.set(-340, 450, 900);   //ovni: vuela, mas arriba
grupoRing.position.set(2000, 0, 0);           //puerta estelar: quieta, al final del eje X
grupoTorus.position.set(700, 200, 900);       //satelite: vuela, un poco mas arriba

grupoCono.scale.set(24, 24, 24);
grupoCilindro.scale.set(24, 24, 24);
grupoRing.scale.set(70, 70, 70);
grupoTorus.scale.set(40, 40, 40);

escena.add(grupoCono);
escena.add(grupoCilindro);
escena.add(grupoRing);
escena.add(grupoTorus);
```

El cohete ($x = -680$), el OVNI (-340) y el satélite (700) parten alineados sobre X, con la misma $z = 900$, y luego se ponen a volar; cada uno a una altura distinta ($y = 150, 450$ y 200). La **puerta estelar** es la única que se queda quieta: está al final del eje X ($x = 2000$, $y = 0$, $z = 0$), más allá de la órbita de Urano, y es la más grande de las figuras ($scale\ 70$). Con $rotation.y = \text{Math.PI} / 2$ queda “de frente en X”: su cara mira a lo largo del eje X, hacia el Sol. Por eso la cámara inicial se colocó al lado del eje X. Después `escena.add(...)` las agrega a la escena: **si no se agrega, no se ve**.

5. El movimiento

5.1 Órbitas

```
controls = new THREE.OrbitControls(camara, renderer.domElement);

//órbitas alrededor del Sol: radio de la órbita y velocidad angular (mas lenta cuanto mas lejos,
ley de Kepler)
function velocidad(radio){
    return 0.005 * Math.pow(510 / radio, 1.5);
}
var orbitas = [
    {cuerpo: Mercurio,      radio: 300,  angulo: 0.5},
    {cuerpo: Venus,        radio: 400,  angulo: 2.0},
    {cuerpo: grupoTierra,  radio: 510,  angulo: 4.0},
    {cuerpo: marte,        radio: 620,  angulo: 1.0},
    {cuerpo: Jupiter,      radio: 900,  angulo: 3.0},
    {cuerpo: grupoSaturno, radio: 1250, angulo: 5.5},
    {cuerpo: Urano,        radio: 1600, angulo: 0.0},
    {cuerpo: grupoCono,    radio: Math.hypot(-680, 900), angulo: Math.atan2(900, -680)},
    {cuerpo: grupoCilindro, radio: Math.hypot(-340, 900), angulo: Math.atan2(900, -340)},
    {cuerpo: grupoTorus,   radio: Math.hypot(700, 900),  angulo: Math.atan2(900, 700)}
];
orbitas.forEach(function(o){
    o.cuerpo.position.x = Math.cos(o.angulo) * o.radio;
    o.cuerpo.position.z = Math.sin(o.angulo) * o.radio;
});
```

Para que algo dé vueltas en círculo alrededor del Sol se usa trigonometría:

```
x = cos(angulo) * radio
z = sin(angulo) * radio
```

Si el `angulo` aumenta un poquito en cada cuadro, el objeto recorre el círculo. `radio` es la distancia al Sol. La función `velocidad(radio)` hace que los lejanos vayan más lentos (como en la realidad: Neptuno tarda mucho más que Mercurio). El cohete, el OVNI y el satélite están en la **misma lista** y dan vueltas igual que los planetas; el OVNI va más alto porque su `y` es 450 (el cohete 150 y el satélite 200) y esa no cambia.

5.2 El ciclo de animación

```
function proyectar(){
  //el Sol permanece completamente quieto en el centro

  orbitas.forEach(function(o){
    o.angulo += velocidad(o.radio);
    o.cuerpo.position.x = Math.cos(o.angulo) * o.radio;
    o.cuerpo.position.z = Math.sin(o.angulo) * o.radio;
  });

  //rotacion propia de la Tierra y orbita lunar
  Tierra.rotation.y += 0.02;
  anguloLuna += 0.03;
  luna.position.set(Math.cos(anguloLuna) * 40, 0, Math.sin(anguloLuna) * 40);

  //la punta del cohete apunta hacia donde avanza
  grupoCono.rotation.y = -orbitas[7].angulo;

  requestAnimationFrame(proyectar);
  renderer.render(escena, camara);
}
requestAnimationFrame(proyectar);
```

1. `requestAnimationFrame(proyectar)` le dice al navegador: “vuelve a ejecutar esta función en el siguiente cuadro” (unas 60 veces por segundo). Eso crea el movimiento.
2. En cada ejecución, cada objeto de la lista `orbitas` suma un poco a su ángulo y se recalcula su posición.
3. La Tierra gira sobre sí misma (`rotation.y`) y la Luna da vueltas con otro ángulo, a distancia 40 de la Tierra.
4. El cohete se orienta con `rotation.y = -angulo` para que su punta mire hacia donde avanza. Antes, `rotation.x = Math.PI / 2` lo acuesta, y `rotation.order = 'YXZ'` hace que primero se acueste y luego gire.
5. El Sol, el asteroide y la puerta **no se tocan** en la función, por eso están quietos.
6. Al final, `renderer.render(escena, camara)` toma la “foto” de ese instante y la dibuja.

6. Las texturas (imágenes)

Archivo	Se usa en
texturaSol1, Mercurio1, Venus1, Tierra1, Luna1, Marte1, Jupiter1, saturno1, AnilloSaturno, Urano1, Ast	Sol, planetas, Luna, anillo y asteroide (originales del proyecto)
texturaCohete, texturaCoheteRojo	Cuerpo y detalles rojos del cohete
texturaOvni, texturaCristal, texturaLuces	Metal del OVNI/satélite/marcos, cúpula y ventanas, aros de luces
texturaCilindro, texturaPanel	Casco con paneles, paneles solares
texturaPortal	Remolino de galaxia de la puerta

Las texturas en negrita fueron creadas a medida para combinar con la estética espacial (colores neón, metal, nebulosas). Para cambiar una, basta reemplazar el archivo en `tex_planetas` manteniendo el nombre.

7. Cómo modificar cosas (guía rápida)

Quiero...	Cambio...
Que un planeta sea más grande	El primer número de su <code>SphereGeometry(...)</code> .
Que un planeta esté más lejos o cerca	<code>radio</code> en la lista <code>orbitas</code> .
Que un planeta vaya más rápido	El <code>0.005</code> en la función <code>velocidad</code> (afecta a todos).
Mover una figura	<code>grupoX.position.set(x, y, z)</code> en la sección “posicionar”.
Agrandar una figura	<code>grupoX.scale.set(n, n, n)</code> .
Quitar algo de la pantalla	Borrar su línea <code>escena.add(...)</code> .
Que una figura deje de moverse	Quitar su línea de la lista <code>orbitas</code> .
Cambiar una imagen	Reemplazar el archivo en <code>tex_planetas</code> .

8. Glosario

Término	Significado
Radián	Unidad para medir ángulos. <code>Math.PI</code> = 180°; <code>Math.PI / 2</code> = 90°.
Segmentos	Cuántos trozos forman una figura curva. Más segmentos = más suave.
Textura	Imagen pegada sobre una figura 3D.
Mesh	Objeto visible: geometría + material.
Group	Contenedor que une varios objetos para moverlos juntos.
Render	Dibujar en pantalla el estado actual de la escena.
Órbita	Trayectoria circular alrededor del Sol.